A FRIENDLY PROJECT GUIDE

CivicEye

How the prototype works, what we learned, and how to explain it

A neighbourhood issue can be easy to miss. CivicEye gives people a simple way to describe one, add a photo and landmark, and see its status on a shared dashboard.

In one sentence: CivicEye turns a resident's issue report into a saved, visible item that the community can follow.

What the prototype can do

- Submit an issue with a short title, category, description, place or landmark, and optional photo.
- Save report details in a small local database and keep photos as files.
- Show reports on a dashboard and let the demo user change status.
- Offer an optional AI summary and priority hint, with a simple local fallback.

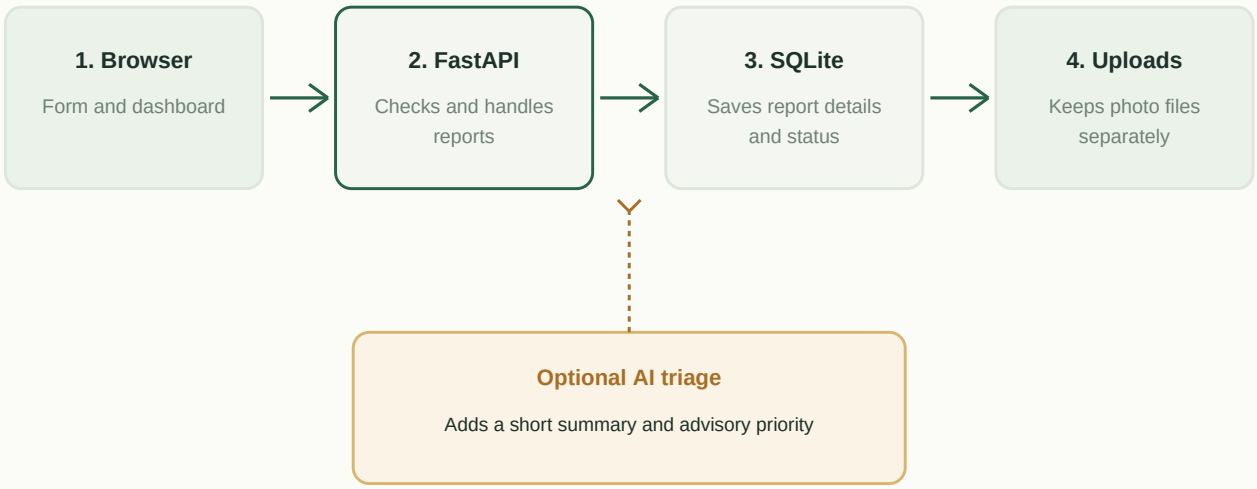
This is a working hackathon prototype. It is not connected to a city department, does not dispatch workers, and cannot confirm that a real-world issue is fixed.

Who this guide is for

Use it to understand the main pieces, restart the app, explain what happens after someone submits a report, and answer common questions during a demo.

1. How the pieces fit together

Think of CivicEye as a short path from a web page to saved information. Each part has one job.



The parts in everyday language

Part	What it does
Browser page	The form and dashboard people see. It sends the entered information to the app and displays the response.
FastAPI app	The Python server behind the page. It checks required fields, saves reports, returns the list, and accepts status changes.
SQLite database	A database stored as one file on this computer. It keeps titles, descriptions, categories, locations, dates, and statuses.
Uploads folder	A folder that holds the optional image files. The database stores the image's name so the page can display it.
Optional AI API	When configured, it receives the report category and description and returns a short summary and advisory priority. It does not receive the photo.

If there is no AI key, report submission still works. CivicEye uses its built-in fallback instead.

2. Technologies and algorithms

A technology is a tool used to build part of the app. An algorithm is a set of steps or rules the app follows.

Main technologies

Technology	Where it is used	Why we chose it
HTML	Page fields, headings, and report cards.	It gives the page its structure.
CSS	Colors, spacing, layout, and mobile view.	It makes the page clear and usable on different screens.
JavaScript	Submitting the form, refreshing reports, and filtering by category or status.	It updates the page without requiring a full manual reload each time.
Python + FastAPI	The server that checks input, saves reports, lists them, and updates status.	It is a quick way to build clear web routes for the prototype.
SQLite	Stores report text and status in civiceye.db.	It needs little setup for a small local demo.
AI API (optional)	Summarizes the category and description and suggests a priority.	It demonstrates how AI could help sort reports, while the app still works without it.

The main algorithms and rules

Rule	In simple terms
Input checks	Before saving, the server checks that required details exist, the category is allowed, and the photo is an accepted type and size. This prevents incomplete or oversized submissions.
Keyword fallback	If no AI key is configured or the AI service fails, the app looks for a few words such as 'flood' or 'exposed wire'. If it finds one, it suggests High priority; otherwise Normal. This is a simple rule, not machine learning.
Optional AI step	When an API key is configured, the app sends only the category and description text, asks for a short summary and priority, and uses a local fallback if the call fails. The AI hint is not proof that a report is true or urgent.
Dashboard filter	The page keeps reports whose category and status match the chosen filters, and hides the rest from the current view.

Presentation shortcut: JavaScript runs the page, Python and FastAPI handle the requests, SQLite remembers reports, and a keyword rule or optional AI suggests a summary and priority.

3. What happens to a report?

Here is the journey from the form to the dashboard.

1	A person fills in the title, category, description, and chosen place. A photo is optional.
2	The app checks the fields and accepts JPG, PNG, or WebP photos up to 8 MB.
3	The report text is summarized by the optional AI service, or by the local fallback when AI is not configured or unavailable.
4	The text details go into civiceye.db. If a photo was attached, its file goes into uploads/.
5	The dashboard reloads the saved report. A demo user can change it between Submitted, In progress, and Resolved.

What the AI hint means

- The hint is only a suggestion. It does not verify an issue or decide which city worker should respond.

- The image is not sent to the AI service; only the selected category and description are sent.

- The local fallback uses a few obvious words such as 'flood' or 'exposed wire' to flag possible urgency. It can miss real hazards or flag something incorrectly.

For an immediate danger, people should contact the appropriate local emergency or civic service. CivicEye is not an emergency reporting channel.

Status counts

Submitted means a resident entered it. In progress means someone changed the demo status to show work is underway. Resolved means someone marked it resolved. The app cannot independently confirm either action.

4. Where things live and how to run it

The project folder contains the app, browser files, report database, and photos.

File or folder	Purpose
app/main.py	Python server, form checks, AI/fallback logic, and database endpoints.
static/	The page design (index.html), styling (styles.css), and browser behaviour (app.js).
civiceye.db	Saved report records. Created automatically when the app starts.
uploads/	Uploaded report photos.
README.md	Setup steps, feature list, demo flow, project plan, and deployment notes.

Start the app in VS Code

Open the CivicEye project folder, choose Terminal > New Terminal, and run these commands one at a time:

```
py -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe -m uvicorn app.main:app --reload
```

Then open http://127.0.0.1:8000 in your browser. Leave the terminal running. Press Ctrl+C in that terminal to stop the server.

Quick fixes

- Browser says it cannot connect: check that Uvicorn says it is running, and use the http://127.0.0.1:8000 address, not the file:/// page.

- A page change is missing: refresh the browser. Uvicorn reloads Python changes; a hard refresh may be needed for browser files.

- An image is rejected: use JPG, PNG, or WebP and keep it under 8 MB.

- AI does not run: the app still works without a key. If configured, make sure the key is set in the terminal or host environment before starting.

5. Challenges we met and how to handle them

These are the real adjustments made while building the prototype, plus the main issue to handle before sharing it online.

Where it showed up	What was confusing or failed	What we did / what to do
Starting point	The workspace had no existing app to continue.	Created a small project from scratch: browser page, Python server, database, and setup guide.
Category field	The first choices were broad, and 'What happened?' was unclear.	Added clearer categories and changed the fields to 'Short issue title' and 'Describe the problem'. Use the closest category; details go in the description.
Dashboard count	The 'being worked on' number included reports that were only Submitted.	Changed it to count only In progress. If counts look stale, refresh and check the report's selected status.
AI setup	No API key was available during the first build.	Kept AI optional and added a local fallback, so the form remains usable. Store any real key in environment settings, never in code.
Public deployment	The database and photos are local files, and there is no sign-in or moderation.	Before a public launch, restrict who can change status and use persistent data storage. Otherwise anyone may edit reports, and some hosts may erase data on restart.

A working local demo is not the same as a safe public service. Keep real personal details out of the demo.

6. Interview questions to practise

These are common questions a judge or teammate may ask. The answers are short on purpose; say them in your own words.

Question	Simple answer
What problem does CivicEye solve?	It gives residents a simple way to report local issues and lets the community see the report status in one place.
Who is it for?	Residents and community groups who want to make local problems visible and follow updates.
What technologies did you use, and why?	HTML, CSS, and JavaScript make the page. Python and FastAPI handle requests. SQLite saves reports with little setup. This is a practical stack for a short prototype.
What does AI do here?	It can summarize the report text and suggest a priority. It is only an advisory hint; it does not check the photo, verify the issue, or dispatch help.
What if the AI service is unavailable?	The app still works. A local keyword rule creates a short summary and a basic priority hint.
How is priority decided?	With AI enabled, the API suggests it from the text. Without AI, a small list of hazard words can suggest High. This simple rule can miss risks, so it is not a safety decision.
Is CivicEye connected to a city office?	No. This is a hackathon prototype. A real connection would need an authorized civic partner and an agreed process.
How do you protect users?	The prototype asks for a place or landmark instead of precise GPS and does not send photos to the AI. It has no sign-in yet, so a public version needs access controls and moderation.
How will you know it has impact?	We would test it with residents and measure whether reports are easy to submit, receive updates, and help people coordinate. We have not yet proved real-world impact.
What would you improve next?	Test with residents, add moderation and protected status changes, use persistent storage, and work with a civic partner before connecting to official services.

A good interview answer is honest about what works today and what still needs real users, stronger protection, and an authorized partner.

7. A simple way to explain CivicEye

You can use this short explanation when someone asks what you built:

> **CivicEye is a prototype for making neighbourhood problems easier to report and follow. A resident adds a short title, issue type, description, landmark, and optional photo. The app saves it and shows it on a shared dashboard. An optional AI service can summarize the text and suggest a priority, but people still control the report status. It is not connected to a government system yet.**

A simple demo order

- Start with a problem: small local issues can be hard to report and hard for neighbours to track.
- Submit a fictional or consented report with a clear category and landmark.
- Show the report on the dashboard. Try the category filter and status menu.
- Explain the AI fallback and be clear that the prototype does not dispatch work or verify a fix.

When to deploy

Aim for about two days before your presentation, after the main features are stable. Test the hosted version, keep the local version as a backup, and avoid making the first deployment on presentation day.

Before sharing a public link, plan for sign-in or admin-only status changes, persistent storage for reports and photos, and safe handling of any AI key. For a judge demo, use limited access and fictional or consented sample data.

Remember

You do not need to describe every programming detail. Explain the problem, show the report journey, say what AI helps with, and be honest about what the prototype does not do yet.

CivicEye - small reports, better streets.